Intro to Algorithms Notes

Based on Cormen et al.

Abdullah Yassine

June 22, 2025

Contents

1 Chapter 2: Getting Started

1.1 Insertion Sort

We begin our studies by first learning the tools and frameworks we will be using in designing and analyzing algorithms. We do this by first describing and solving the **sorting problem**.

Definition 1.1. *Sorting Problem*

Sorting Problem Definition

input: A sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$
output: A permuation or reordering $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \ldots \leq a'_n$

Definition 1.2. *The* ***keys*** *are the numbers that we wish to sort.*

The **pseodo-code** for the **insertion-sort** algorithmn is provided below. You can find the python implementation in the following path `./ch3.ipynb` of the current working directory.

Algorithm 1 INSERTION-SORT(A)

Require: Array $A[1 \ldots n]$
Ensure: Sorted array A in non-decreasing order

```
for j = 2 to n do
    key ← A[j]
    i ← j − 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]                ▷ Shift element right
        i ← i − 1
    end while
    A[i + 1] ← key                     ▷ Insert key in position
end for
```

You can see that the algorithm first assumes that the first index is sorted. It then takes the next index (just like how you would take the next card out of the cards you have) and sort it in the right place by placing the card in the appropriate place. It does this for all the cards.

As you can see, the insertion-sort algorithm sorts the list **in-place**.

1.2 Algorithms as a Technology

Different solutions to the same problem have different efficiencies in space used and time spent solving the problem. We will learn more about these when we discuss sorting algorithms and how they all differ in terms of space and time complexity.